



UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

CARRERA INGENIERÍA ELECTRÓNICA

**PROYECTO INTEGRADOR PARA LA OBTENCIÓN DEL
TÍTULO DE GRADO INGENIERO ELECTRÓNICO**

**“DISEÑO, DESARROLLO E IMPLEMENTACIÓN DE UN
SOFTWARE PARA LA ADQUISICIÓN Y ANÁLISIS DE ONDAS
DE IMPULSO TIPO RAYO (1,2/50 μ s).”**

Alumno

Tardón, Damián David

Director

Ing. Blasco, Marcos Javier

Co-Director

Ing Serra, Gabriel Horacio

Córdoba, República Argentina

Abril / 2026

Capítulo 1

Adquisición de datos y control de Instrumentos

Este capítulo establece la transición entre los fundamentos teóricos del muestreo y retención analizados previamente, y su implementación práctica en el *software* desarrollado. El objetivo principal radica en la captura y el análisis automatizado de ondas de impulso tipo rayo, parametrizadas bajo la relación normalizada de $1,2/50\text{ }\mu\text{s}$, empleando el *hardware* específico del osciloscopio de almacenamiento digital GW Instek GDS-1102A-U.

1.1 Controlador del osciloscopio

La abstracción del *hardware* se diseña bajo el paradigma de Programación Orientada a Objetos (POO), encapsulando la lógica de control del osciloscopio dentro de la clase `GWInstekGDS1000AU`. Esta estructura modular oculta la complejidad sintáctica del estándar IEEE 488.2 y los comandos SCPI específicos del fabricante. El controlador simplifica y gestiona las operaciones a acciones de consulta (*queries*) y escritura (*writes*) para configurar el instrumento.

De esta forma, tal como se vió en el capítulo **Arquitectura del sistema**, se logra abstraer el *software* del instrumento físico, por lo que, en otro momento se puede utilizar otro modelo de osciloscopio del mismo u otro fabricante, sin tener que modificar el sistema principal, solamente reescribiendo el código interno de cada función de este módulo.

1.2 Comunicación VISA

A continuación, se presenta la implementación de la capa de comunicación entre la computadora y el osciloscopio utilizando la biblioteca PyVISA, que proporciona una interfaz de alto nivel para interactuar con instrumentos de medición a través de diversos buses (USB, GPIB, Ethernet, RS-232, COM, etc.). Cabe aclarar que, si bien el equipo utilizado se conecta a través de un puerto USB, la computadora lo detecta como un dispositivo de comunicación serie (COM) mediante el protocolo USB CDC-ACM, tal como indica el fabricante en su manual de programación [X].

Esta implementación asegura la portabilidad multiplataforma del software sin depender de librerías vinculadas a sistemas operativos específicos o drivers propietarios.

1.2.1 Inicialización y gestión de recursos (`__init__`)

El método constructor `__init__` gestiona la inicialización de las variables de estado. Su propósito es definir un estado inicial seguro al administrador del instrumento, y la instanciación del gestor de recursos. Acto seguido, ejecuta un escaneo automatizado para construir un arreglo con los identificadores de los dispositivos presentes en el canal y evalúa su longitud. Si la detección es exitosa y se encuentra al menos un dispositivo direccionable, el método invoca automáticamente la rutina de conexión sobre el primer elemento del vector de recursos.

```
def __init__(self):
    self.dso = None
    self.rm = pyvisa.ResourceManager('@py')
    instrument_list = self.rm.list_resources()
    print("Instrumentos encontrados:", instrument_list)
    if instrument_list:
        self.connect(instrument_list[0])
```

- **.dso = None:** Inicializa con valor nulo el atributo que contendrá el administrador del dispositivo físico cuando se establezca la conexión.
- **.ResourceManager('@py'):** Instancia el gestor de recursos de VISA usando el *backend* nativo de Python puro (PyVISA-py).
- **.list_resources():** Escanea los buses del sistema para identificar los recursos físicos direccionables disponibles en el entorno local.
- **.connect(instrument_list[0]):** Estructura de control condicional que, ante la presencia de al menos un dispositivo compatible en la lista, toma el primer elemento del arreglo y desvía el flujo hacia el método de conexión.

1.2.2 Establecimiento del enlace bidireccional (connect)

El método `connect` implementa la apertura de la sesión de comunicación con el instrumento seleccionado. Su función es asociar el identificador del recurso físico al administrador del objeto, y configurar los caracteres de terminación de lectura y escritura mediante la constante de salto de línea, para evitar condiciones de bloqueo por tiempo de espera (*timeout*).

Una vez configurado el canal de entrada y salida, ejecuta una validación de la integridad bidireccional del canal transmitiendo una consulta estándar de identificación, y captura los metadatos devueltos por el *firmware*. Todo el flujo analítico se encapsula en una rutina de control de excepciones que, ante fallos críticos en el bus o pérdidas de sincronismo, aborta la operación y libera los recursos del sistema de forma segura, ya que de lo contrario impediría la reconexión posterior del instrumento, debiendo reiniciar la computadora o el osciloscopio.

```
def connect(self, resource_name):
    try:
        self.dso = self.rm.open_resource(resource_name)
        self.dso.read_termination = '\n'
        self.dso.write_termination = '\n'
        idn = self.dso.query('*IDN?')
        print("Instrumento conectado satisfactoriamente:", idn)
    except Exception as e:
        print("Error al iniciar con el instrumento:", e)
    self.close()
```

- **.open_resource(resource_name):** Abre el recurso VISA especificado y asigna la interfaz de entrada/salida (I/O) al objeto del administrador del instrumento.
- **.read_termination** y **.write_termination:** Definen la secuencia de salto de línea (' \n ') como carácter de terminación de mensajes de la capa de transporte, tal como indica el fabricante en el manual de programación del instrumento [X]. Esto previene condiciones de bloqueo crítico o errores por *timeout*.
- **.query('*IDN?'):** Envía la consulta de identificación estándar IEEE 488.2 mediante una operación combinada de escritura y lectura. La correcta recepción del *string* de datos (fabricante, modelo, número de serie y versión de *firmware*) actúa como validación funcional del enlace lógico bidireccional.
- **Bloque de control de excepciones (try/except):** Garantiza el manejo seguro de fallos. Ante un error en la apertura del puerto o incompatibilidad en el protocolo, se intercepta la excepción, se reporta la anomalía en la consola de depuración y se invoca el método **.close()** para liberar los recursos comprometidos.

1.3 Adquisición y decodificación de datos

La transferencia de datos de la memoria del osciloscopio hacia la computadora, se realiza mediante un procedimiento secuencial para garantizar la integridad de los datos. Este proceso de adquisición se estructura en cuatro etapas fundamentales.

Paso 1: Sincronismo y solicitud. Tras confirmar el disparo, se consulta el estado de disponibilidad del canal. Si la forma de onda está lista en la memoria del instrumento, se emite la petición de transferencia de datos.

Paso 2: Decodificación del Encabezado. El flujo de datos inicia con una cabecera de 10 bytes, que el algoritmo decodifica para calcular la longitud neta del paquete de datos, permitiendo un control preciso del bucle de lectura secuencial por bloques y previniendo el truncamiento de la información.

Paso 3: Desempaquetado. Los datos binarios almacenados en la memoria RAM del sistema se decodifican para obtener el periodo de muestreo y los datos crudos (*raw data*) de la forma de onda.

Paso 4: Conversión a magnitud real. Se realiza una transformación lineal para convertir los niveles discretos del ADC a magnitudes de tensión reales, aplicando la escala vertical del canal y la constante de cuantización del instrumento.

1.3.1 Adquisición de datos (`get_block_data`)

El método `get_block_data` tiene como función principal coordinar y ejecutar la adquisición completa del flujo binario de datos correspondiente a la memoria del canal seleccionado del osciloscopio. Su función es verificar el estado de preparación de la señal en el instrumento, solicitar la transferencia del bloque de memoria, interpretar las dimensiones físicas del paquete mediante la lectura de su cabecera estándar e implementar un ciclo de lectura iterativo y fragmentado en bloques de *bytes* para reconstruir la trama binaria total en la memoria RAM del ordenador de forma segura.

```
def get_block_data(self, channel):
    try:
        v_div = self.get_channel_scale(channel)
        self.dso.write(f':acquire{channel}:state?')
        state = self.dso.read()
        if state[0] == '1':
            time.sleep(0.1)
            self.dso.write(f':acquire{channel}:memory?')
            inBuffer = self.dso.read_bytes(10)
            length = len(inBuffer)
            headerlen = 2 + int(chr(inBuffer[1]))
            pkg_length = int(inBuffer[2:headerlen]) + headerlen
            pkg_length = pkg_length - length
            while True:
```

```
        if pkg_length == 0:
            break
        else:
            if pkg_length > 100000:
                length = 100000
            else:
                length = pkg_length
            try:
                buf = self.dso.read_bytes(length)
            except:
                print('Error al recibir datos del instrumento!')
                self.close()
                sys.exit(0)
            num = len(buf)
            inBuffer += buf
            pkg_length = pkg_length - num
            waveform, dt = self.unpack_waveform(inBuffer, headerlen, v_div)
            return inBuffer, waveform, dt
    else:
        print('Error: Forma de onda aún no está lista.')
        return None, None, None
except Exception as e:
    print("Error al obtener datos:", e)
    self.close()
    return None, None, None
```

- **.get_channel_scale(channel):** Consulta la escala vertical del canal seleccionado.
- **.write(f':acquire{channel}:state?'):** Consulta el estado de adquisición del canal especificado. Si el primer carácter del *string* devuelto equivale a '1', se confirma que el osciloscopio ha finalizado la captura, y que los datos en memoria se encuentran listos para su transferencia.
- **time.sleep(0.1):** Introduce un retardo de 0,1 s para estabilizar las transferencias del bus USB, tras la confirmación del estado operativo.
- **.write(f':acquire{channel}:memory?'):** Solicita la transferencia de datos para el canal seleccionado.
- **.read_bytes(10):** Ejecuta una lectura binaria de los primeros 10 bytes para procesar el encabezado.
- **headerlen:** Calcula la longitud de la cabecera.
- **inBuffer[1]:** Indica cuántos dígitos representan el tamaño del paquete.
- **pkg_length:** Declara la cantidad neta de *bytes* de la forma de onda.

- **Ciclo while:** Se ejecuta mientras el valor residual de `pkg_length` sea mayor a cero. Para evitar la saturación de las colas de comunicación del *backend* y controlar el uso de memoria, se limita el tamaño de cada lectura a 100 000 bytes por ciclo. Los datos parciales leídos mediante `.read_bytes(length)` se concatenan secuencialmente en el contenedor binario `inBuffer`, disminuyendo el valor de `pkg_length` de acuerdo a la longitud del bloque recibido, hasta vaciar el *buffer* del osciloscopio. En caso de producirse un error de lectura, se captura la excepción, se interrumpe el proceso mediante `sys.exit(0)` y se liberan las sesiones activas del bus.

1.3.2 Decodificación de datos (`unpack_waveform`)

El método `unpack_waveform` cumple la función de decodificar el *buffer* de bytes brutos acumulados, transformándolos en arreglos numéricos de magnitud física real. Este método procesa secuencialmente los segmentos del bloque binario mediante operaciones de desempaquetado (*unpacking*) bajo formatos de ordenación de *bytes* específicos, aislando el periodo de muestreo y el arreglo de datos crudos (*raw data*) del ADC, para finalmente aplicar la constante de discretización vertical del instrumento y retornar la forma de onda expresada directamente en Volts.

```
ADC_STEPS_PER_DIV = 25.0
def unpack_waveform(self, inBuffer, headerlen, vdiv):
    print(inBuffer[:headerlen])
    dt = unpack('>f', inBuffer[headerlen : headerlen + 4])[0]
    print(f'Periodo de muestreo = {dt*1e9:.0f} [ns]')
    waveform_raw = unpack('>%sh' % (int(len(inBuffer[headerlen + 8:]) / 2)),
                          inBuffer[headerlen + 8:])
    waveform_raw = np.array(waveform_raw)
    num = len(waveform_raw)
    print(f'Cantidad de muestras = {num}')
    waveform = waveform_raw * vdiv / self.ADC_STEPS_PER_DIV
    return waveform, dt
```

- **Desempaquetado del periodo de muestreo (dt):** La función `unpack` analiza los cuatro *bytes* posteriores a la cabecera como un número de coma flotante de precisión simple de 4 bytes (32 bits) en formato (*big-endian*) (expresado como: '>f'), el cual representa el periodo de muestreo en segundos.
- **Desempaquetado de la señal cruda (waveform_raw):** La cantidad (s) de *bytes* restantes componen la señal digitalizada, como números enteros cortos con signo en formato *big-endian* (expresado como: '>%sh'). Dado que cada muestra ocupa 2 bytes (16 bits), la cantidad (s) a desempaquetar se calcula como la mitad de la longitud del bloque binario.
- **np.array(waveform_raw):** Convierte el arreglo bruto (`waveform_raw`) en un arreglo optimizado de NumPy, para facilitar su manipulación.

- **Escalado de la señal:** Se aplica la Ecuación 1.1, multiplicando el arreglo crudo por la escala vertical (V/div), y dividiéndolo por la constante de 25 pasos discretos por división vertical, para transformar los niveles discretos del ADC a magnitudes de tensión reales.

$$\text{waveform}(i) = \frac{\text{waveform_raw}(i) \cdot \text{V/div}}{\text{ADC_STEPS_PER_DIV}} \quad (1.1)$$

1.4 Configuración del instrumento

Además de las rutinas de inicialización y transferencia de datos, el módulo integra diversas funciones específicas utilizadas por el presentador del *software* para sincronizar la interfaz gráfica con el osciloscopio. A continuación, se listan los métodos empleados y su propósito en el programa:

- **default_settings:** Transmite el comando estándar ***RST** para restablecer el instrumento a su configuración de fábrica, garantizando un estado inicial conocido antes de aplicar los nuevos parámetros del ensayo.
- **set_channel_scale / get_channel_scale:** Modifican y consultan la escala de tensión por división del canal indicado.
- **set_channel_offset / get_channel_offset:** Controlan el nivel de tensión continua de referencia (*offset*) sobre el cual se centra la señal en la pantalla para el canal indicado.
- **set_timebase_scale / get_timebase_scale:** Ajustan y leen la escala horizontal de tiempo por división. Implícitamente define la frecuencia de muestreo.
- **set_timebase_position / get_timebase_position:** Mueve el origen temporal de la señal respecto a la posición central de la pantalla.
- **set_trigger_level / get_trigger_level:** Configuran el umbral de tensión que la señal debe cruzar para iniciar a "dibujar" o sincronizar la onda en la pantalla.
- **set_trigger_slope / get_trigger_slope:** Definen la polaridad del flanco de disparo, determinando si la captura se activa en la rampa ascendente o descendente del impulso.
- **set_single_trigger:** Arma el osciloscopio en modo de adquisición de un único evento, dejándolo en espera hasta que aparezca la señal.
- **get_trigger_state:** Consulta el estado interno del barrido para detectar cuándo el instrumento pasa de estar «No disparado» a estar «Disparado».

Al analizar el código fuente de este módulo, se puede observar que hay otros métodos implementados, pero que no se utilizan en el programa principal. Esto se debe a que fueron desarrollados para futuras funcionalidades, pero no forman parte del flujo actual de adquisición y visualización de datos. Sin embargo, su presencia en el código no afecta el funcionamiento del sistema, ya que no son invocados por ningún proceso activo.

1.4.1 Métodos de acceso y modificación

Para gestionar la configuración del osciloscopio de manera robusta, los métodos recién mencionados en la sección 1.4 implementan una arquitectura estandarizada, basada en métodos de acceso (*getters*) y métodos de modificación (*setters*), en la clase **GWInstekGDS1000AU**.

Los métodos *getters* encapsulan la directiva SCPI de consulta mediante la instrucción `.query()`. Su función es interrogar al instrumento sobre su estado actual, recibir la cadena de caracteres devuelta por el *firmware*, convertirla al tipo de dato correspondiente (flotante, entero o cadena) y retornar dicho valor a la interfaz, como se observa en el siguiente ejemplo para leer la escala vertical de un canal.

```
def get_channel_scale(self, channel):
    try:
        scale = self.dso.query(f':channel{channel}:scale?')
        v_scale = float(scale)
        print(f'Escala vertical canal {channel}: {v_scale:.2f} [V/div]')
        return v_scale
    except Exception as e:
        print("Error al obtener la escala vertical:", e)
        self.close()
```

Por su parte, los métodos *setters* son los responsables de modificar los parámetros del equipo. Para garantizar la consistencia entre el *software* y el *hardware*, operan bajo un esquema de lazo cerrado:

1. Transmiten el comando de configuración mediante la instrucción `.write()`.
2. Invocan internamente al método *getter* asociado para leer el valor que el osciloscopio efectivamente adoptó.
3. Comparan el valor deseado con el valor leído e informan si la acción se concretó con éxito o si el instrumento rechazó la configuración.

Tal como se muestra en el siguiente ejemplo para configurar la escala vertical de un canal.

```
def set_channel_scale(self, channel, value):
    try:
        self.dso.write(f':channel{channel}:scale {value}')
        v_scale = self.get_channel_scale(channel)
        if v_scale == value:
            print(f'Escala vertical canal {channel} configurada a: {v_scale:.2f} [V/div]')
        else:
            print('No se pudo configurar la escala vertical.')
    except Exception as e:
        print("Error al configurar la escala vertical:", e)
        self.close()
```

Ambas familias de métodos se encuentran encapsuladas dentro de bloques de control de excepciones (**try/except**), que ante cualquier pérdida de comunicación, desbordamiento del *buffer* o error de sintaxis, el algoritmo captura la anomalía, notifica el fallo e invoca inmediatamente al método **.close()**, para asegurar que los recursos del sistema operativo y los administradores del bus USB se liberen de forma segura, previniendo el bloqueo lógico del puerto y evitando fugas de memoria o congelamientos del instrumento.

1.5 Cierre de conexiones y liberación de recursos (close)

El método **close** tiene como función principal centralizar la desasignación y el cierre formal de las sesiones lógicas y físicas abiertas en el bus de comunicación con el osciloscopio. Su propósito es mitigar fallos por bloqueo de puertos o saturación de *buffers* en el *hardware* mediante la terminación explícita del enlace con el instrumento y la posterior destrucción de la instancia del administrador de recursos de VISA, garantizando que el sistema operativo recupere los descriptores de archivos lógicos asociados al puerto USB independientemente de si el programa finalizó con éxito o por una interrupción abrupta.

```
def close(self):
    if self.dso:
        try:
            self.dso.close()
            print("Conexión con el instrumento cerrada exitosamente.")
        except Exception as e:
            print("Error al cerrar la conexión con el instrumento:", e)
        finally:
            self.dso = None
    if hasattr(self, 'rm') and self.rm is not None:
        try:
            self.rm.close()
            print("Gestor de recursos cerrado exitosamente.")
        except Exception as e:
```

```
print("Error al cerrar el gestor de recursos:", e)
```

- **Evaluación y cierre del instrumento (`if self.dso`):** Se verifica si el atributo `.dso` contiene un recurso abierto. De ser así, se introduce un bloque de seguridad `try/except/finally` donde se invoca explícitamente a `.dso.close()` para enviar la directiva de desconexión al bus. El bloque `finally` fuerza la reasignación de `.dso` a un estado nulo (`None`), asegurando que el canal se encuentre cerrado.
- **Destrucción del gestor de recursos (`hasattr(self, 'rm')`):** Se comprueba de forma segura si el objeto posee asignado el atributo del gestor de recursos lógicos y que este no es nulo. Bajo una estructura protegida `try/except`, se invoca al método `.rm.close()`, lo que destruye la instancia del gestor de recursos PyVISA y libera por completo el direccionamiento de los buses.

1.6 Destructor de la instancia (`__del__`)

El método especial `__del__` actúa como el destructor de la clase, teniendo como función principal asegurar la ejecución automática de las rutinas de limpieza cuando el ciclo de vida del objeto llega a su fin o la instancia es removida de la memoria RAM por el recolector de basura (*garbage collector*) de Python. Su única instrucción consiste en invocar directamente al método `self.close()` (perteneciente a la biblioteca PyVISA), lo que constituye un mecanismo de seguridad redundante frente a fallos lógicos o descuidos de programación, garantizando de forma determinista que ningún canal físico o puerto lógico USB permanezca bloqueado o inaccesible tras la finalización del proceso analítico.

```
def __del__(self):
    self.close()
```

Al destruirse la instancia por pérdida de ámbito o finalización del *script* principal, el entorno de ejecución de Python llama a esta función. Al ejecutar `self.close()`, se garantiza de forma transparente para el programador la desconexión segura del enlace de datos.

1.7 Conversor de unidades (`process_multipliers`)

El método estático `process_multipliers` tiene como función realizar la conversión de parámetros ingresados desde la interfaz gráfica, asociando una magnitud numérica a su correspondiente prefijo del Sistema Internacional de Unidades (SI). Su propósito es actuar como una capa analítica que procesa cadenas de texto provenientes de

selectores, obteniendo el factor multiplicador correspondiente para estandarizar las magnitudes en variables de coma flotante con unidades base (volts y segundos), tal como requiere la sintaxis gramatical SCPI del osciloscopio.

A diferencia de los otros métodos, este se define como *estático* (@staticmethod), lo que implica que no depende del estado interno de la clase ni de sus instancias. Esto permite que pueda ser invocado directamente desde la clase sin necesidad de crear un objeto, facilitando su uso en cualquier parte del programa para convertir unidades de forma rápida.

```
@staticmethod
def process_multipliers(value, unit):
    multipliers = {
        "V": 1.0,          # Volt.
        "mV": 1e-3,        # Milivolt.
        "uV": 1e-6,        # Microvolt.
        "s": 1.0,          # Segundo.
        "ms": 1e-3,        # Milisegundo.
        "µs": 1e-6,        # Microsegundo.
        "ns": 1e-9         # Nanosegundo.
    }
    try:
        numerical_value = float(value)
        factor = multipliers.get(unit, 1.0)
        scale = numerical_value * factor
        return scale
    except ValueError:
        return None
```

- **Diccionario de submúltiplos (multipliers):** Define un mapa de constantes técnicas indexado por cadenas de texto que representan las unidades de medida y sus prefijos normalizados. Almacena las relaciones escalares para volt (V: 1.0), milivolt (mV: 10^{-3}), segundo (s: 1.0), milisegundo (ms: 10^{-3}), microsegundo (µs: 10^{-6}) y nanosegundo (ns: 10^{-9}).
- **Conversión numérica (float(value)):** Convierte (*casting*) la cadena de texto de entrada (recibida de la interfaz gráfica) a un tipo de dato numérico de coma flotante.
- **Determinación del factor de escala (multipliers.get(unit, 1.0)):** Recupera del diccionario la constante matemática asociada a la unidad de entrada mediante el método get; en caso de recibir una unidad no indexada, adopta de manera predeterminada un factor neutro unitario (1.0).
- **Cálculo de magnitud final:** Multiplica el valor flotante obtenido, por su factor de escala para obtener el parámetro real final, retornando el resultado para su posterior inyección en la cadena del comando SCPI.

- **Manejo de excepciones:** Todo el bloque se encapsula en una estructura **try/except ValueError**. Intenta realizar el *casting* explícito de la cadena de texto de entrada a un tipo de dato flotante numérico. Si la entrada de texto no corresponde a un formato numérico válido, se captura la excepción `ValueError`, y se retorna un puntero nulo (`None`) para detener de forma segura la transmisión hacia el bus.